Ex. No. 11	ADD iOS AS A BUILD TARGET
Date of Exercise	
Git Hub Host Link	https://github.com/gurubruhh/Flutter-Laboratory.git

Aim

To configure and build a Flutter application for ios.

Description

This experiment demonstrates developing a cross-platform application for iOS using Flutter. The project is configured and opened using Xcode. The application is executed on an iOS simulator and tested for UI functionality, navigation and interactions. The appropriate iOS build process is also explored..

Procedure:

1. Check Flutter installation using flutter doctor.
2. Check available devices using flutter devices.
3. Create a project using flutter create ios_app.
4. Open the project in VS Code.
5. Open the iOS workspace using open ios/Runner.xcworkspace.
6. Select an iOS simulator in Xcode.
7. Design the application interface using Flutter widgets.
8. Run the application using flutter run.
9. Test the UI, navigation, text fields, buttons and images.
10. Build the application using the appropriate Xcode build process.

Program

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'iOS Application',
      theme: ThemeData(
```

```
primarySwatch: Colors.blue,
      ),
      home: const HomeScreen(),
    );
  }
}

class HomeScreen extends StatelessWidget {
  const HomeScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('iOS Application'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ),

      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Icon(
              Icons.phone_iphone,
              size: 100,
              color: Colors.blue,
            ),

            const SizedBox(height: 20),

            const Text(
              'Welcome to iOS App!',
              style: TextStyle(
                fontSize: 24,
                fontWeight: FontWeight.bold,
              ),
            ),

            const SizedBox(height: 10),

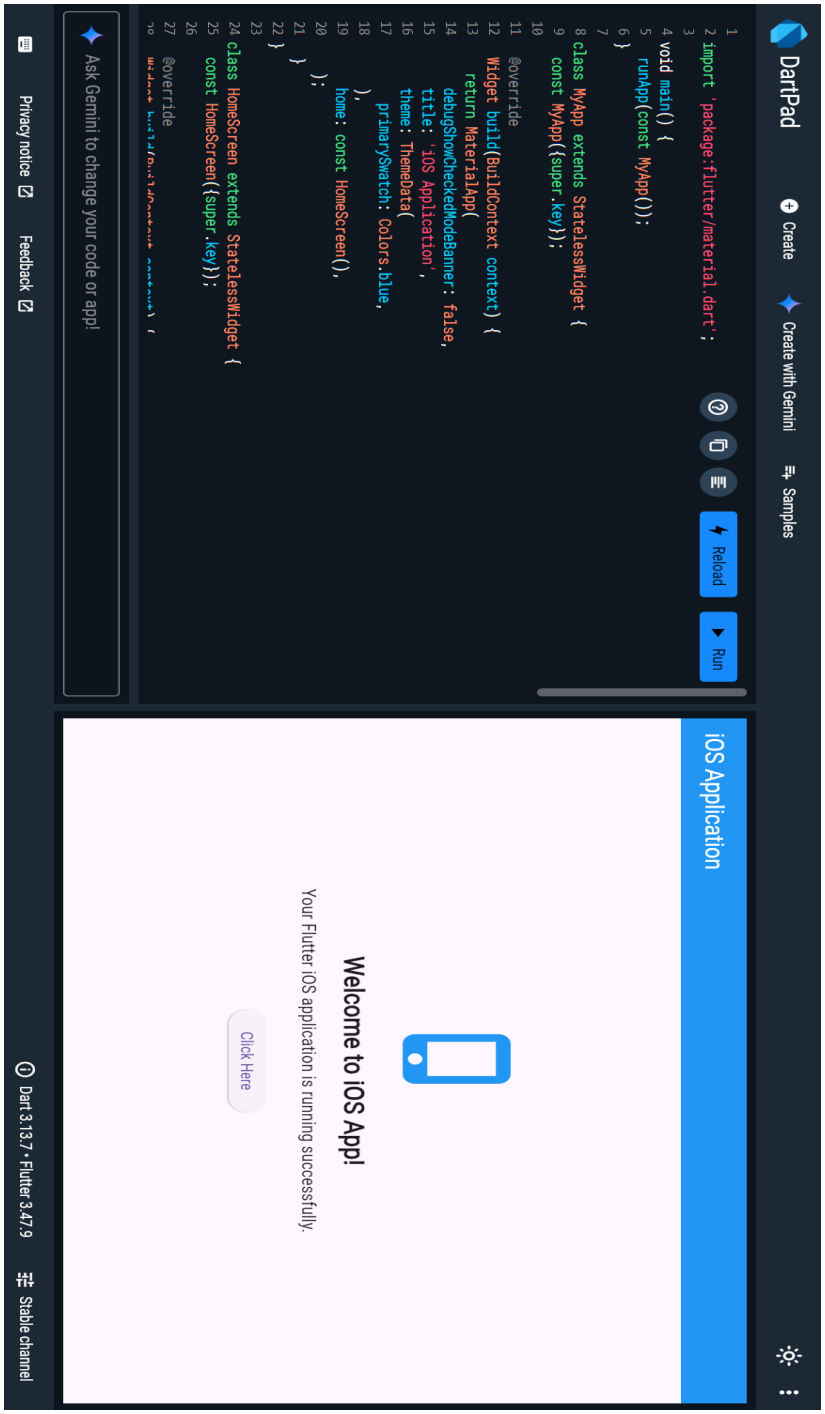
            const Text(
              'Your Flutter iOS application is running successfully.',
              textAlign: TextAlign.center,
              style: TextStyle(fontSize: 16),
            ),

            const SizedBox(height: 25),

            ElevatedButton(
```

```
onPressed: () {
              ScaffoldMessenger.of(context).showSnackBar(
                const SnackBar(
                  content: Text('Button clicked successfully!'),
                ),
              );
            },
            child: const Text('Click Here'),
          ),
        ],
      ),
    ),
  );
 }
}
```

Output


DartPad
Create
Create with Gemini
Samples
Reload
Run
import 'package:flutter/material.dart';
void main() {
runApp(const MyApp());
}
class MyApp extends StatelessWidget {
const MyApp({super.key});
@override
Widget build(BuildContext context) {
return MaterialApp(
debugShowCheckedModeBanner: false,
title: 'iOS Application',
theme: ThemeData(
primarySwatch: Colors.blue,
),
home: const HomeScreen(),
);
}
}
class HomeScreen extends StatelessWidget {
const HomeScreen({super.key});
@override
Ask Gemini to change your code or app!
Privacy notice
Feedback
iOS Application
Welcome to iOS App!
Your Flutter iOS application is running successfully.
Click Here
Dart 3.13.7 • Flutter 3.47.9
Stable channel

Result

Thus, the Flutter applications were successfully developed, executed, and tested on the respective platforms. The required functionalities, user interface, and responsive layouts were verified successfully.